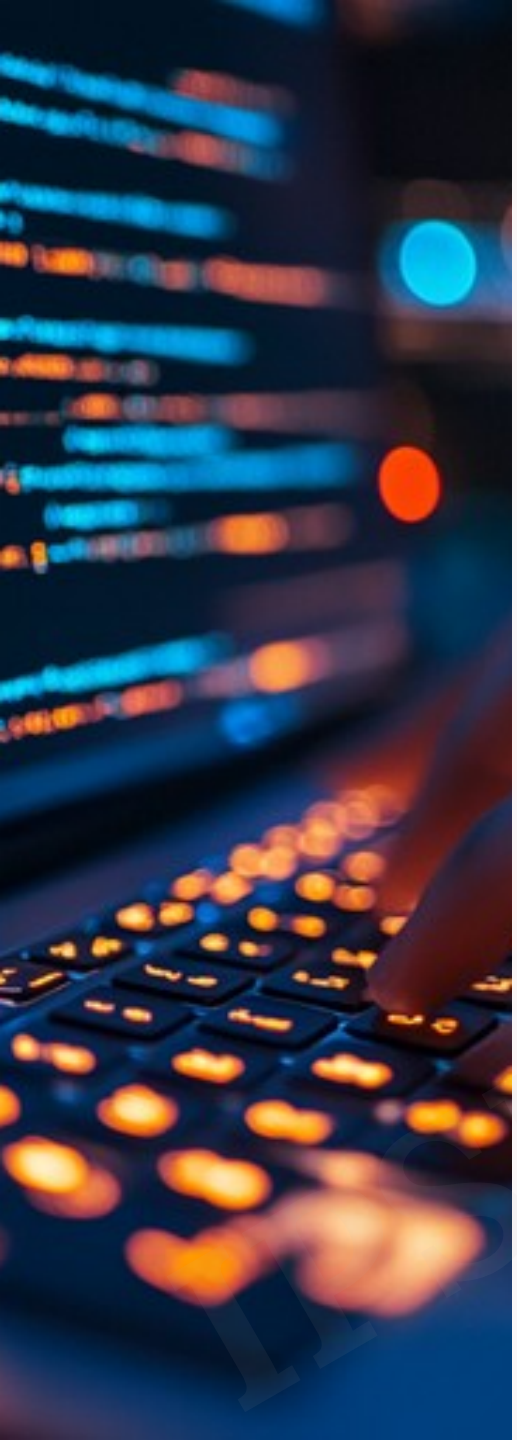


MSYS2 & Arm GNU Toolchain

李建緯
雲科大資工系 智慧感知與系統整合實驗室
IPSI Lab, Yuntech-CSIE

IPSI Lab, Yuntech-CSIE

Arm GNU Toolchain — GDB

Arm GNU Toolchain 中的 GDB

- **GDB（GNU Debugger）** 是 GNU 工具鏈的一部分，專門用於 **程式偵錯**
- 此套 `arm-none-eabi-gdb` 可解析 **AArch32 的 ARM（A32） / Thumb** 指令及 ARM926EJ-S 的暫存器；本次 `versatilepb` 實驗**不執行 AArch64 / A64 指令**。
- 常用於 **嵌入式系統開發**，適合 ARM 處理器的偵錯需求

IPSI Lab, Yuntech-CSIE

為什麼使用 Arm GDB？

1. **支援 AArch32 指令集**：可除錯傳統 ARM 與 Thumb 指令；本課程模擬板為 **ARM926EJ-S（Armv5TE）**。其他架構應確認 GDB 與 QEMU 目標相容。
2. **交叉編譯支援**：可在 **x86 主機**上使用 ARM GDB 來偵錯目標 ARM 裝置，方便嵌入式開發
3. **遠端偵錯**：結合 **GDB Server** 可進行遠端偵錯
4. **硬體級偵錯**：支援 **J-Link**、**OpenOCD** 等硬體除錯工具

IPSI Lab, Yuntech-CSIE

常用 GDB 指令

請分清楚： `info registers r0` **讀暫存器數值；** `x/wx $r0` **則把 R0 當成記憶體位址。前者適用整數參數，後者適用合法的指標參數。**

完整指令	縮寫	功能
`break <location>`	`b`	**設置中斷點**：暫停程式執行於指定位置，可在函數、檔案行號或記憶體位址處設置。用於排查特定邏輯或指令執行前後的狀態。
`continue`	`c`	**繼續執行程式**：從中斷點或暫停處繼續執行程式。適合在多個中斷點間快速執行，檢查跳躍執行的行為是否符合預期。

IPSI Lab, Yuntech-CSIE

完整指令	縮寫	功能
`next`	`n`	**逐步執行，但不進入函數**：適用於高階語言的程式碼，對於函數呼叫直接跳過。用於快速檢查程式流程而不深入子函數內部細節。
`step`	`s`	**逐步執行，並進入函數**：適用於高階語言，若當前行包含函數呼叫，會跳轉到函數內部逐步執行。用於分析特定函數的執行細節。
`stepi`	`si`	**單步執行一條機器指令**：適用於組合語言或機器指令級別的調試，可逐條指令檢查暫存器與記憶體的變化，對低階偵錯尤為重要。

IPSI Lab, Yuntech-CSIE

完整指令	縮寫	功能
`print <variable>`	`p`	**查看變數內容**：適用於高階語言，顯示指定變數當前的值。對於結構體或陣列，也可以以分層方式顯示成員內容，便於檢查複雜資料結構的狀態。
`x/<format> <address>`	無縮寫	**查看記憶體內容**：以指定格式（如字節、字、雙字）顯示記憶體的內容。適合檢查暫存器指向的記憶體位址，或者調查特定變數儲存位置的資料是否正確。

IPSI Lab, Yuntech-CSIE

完整指令	縮寫	功能
`info registers`	`i r`	**查看所有暫存器的內容**：列出處理器當前暫存器值，適用於組合語言調試或確認函數參數和返回值（如 ARM 中 `r0` 為返回值暫存器）。
`info locals`	無縮寫	**查看當前函數的區域變數**：顯示當前執行範圍內的所有區域變數及其值。對於高階語言函數調試非常有用，可快速檢查變數狀態是否正確。

IPSI Lab, Yuntech-CSIE

完整指令	縮寫	功能
`info frame`	無縮寫	**檢查目前堆疊框架**：顯示框架識別、儲存的 PC、Caller/Callee 資訊等（視除錯資訊和目標而定）；**Arm 並不保證每個函式都有固定 BP 暫存器**。
`backtrace`	`bt`	**顯示呼叫堆疊**：列出程式執行的函數呼叫鏈，顯示各層函數的位置和參數。適合分析程式的執行路徑，找出執行錯誤的原因。
`delete <breakpoint>`	`d`	**刪除指定的中斷點**：用於清除一個或多個不再需要的中斷點，保持執行流程簡潔。可以搭配 `info breakpoints` 確認中斷點編號後使用。

IPSI Lab, Yuntech-CSIE

完整指令	縮寫	功能
`list <location>`	`l`	**顯示程式碼**：列出指定行號或函數的高階語言程式碼，適合快速檢查執行流程中的上下文。用於高階語言檔案的代碼導航。
`watch <expression>`	無縮寫	**設定變數監視點**：程式執行期間，若指定表達式（如變數值）發生變化，會自動暫停程式。適合檢查特定變數是否被意外修改。
`quit`	`q`	**退出 GDB**：結束 GDB 調試環境，退出當前會話。

IPSI Lab, Yuntech-CSIE

完整指令	縮寫	功能
`run`	`r`	**本機 / 支援啟動的目標**可用於開始程式；本課程使用 QEMU `-S -s` 並以 `target remote :1234` 連線，主要使用 `continue` / `stepi`，**不可假設 `run` 能重新啟動此遠端目標**。
`disassemble <function>`	`disas`	**反組譯指定函數**：將二進位程式碼翻譯成組合語言，顯示特定函數或程式位置的組合語言指令，適合檢查編譯器生成的程式碼是否正確。
`display /i $pc`	無縮寫	**顯示程式計數器（PC）當前指令**：每次執行後自動顯示 PC 對應的指令，適合追蹤程式執行的指令流。

IPSI Lab, Yuntech-CSIE

使用範例

先確認環境： 下列 GDB 指令的實際操作，以後面「QEMU `-S -s` → `arm-none-eabi-gdb` → `target remote :1234`」為準。此處為**指令速查**，不是獨立的可執行命令序列。

1. 編譯程式並加入除錯資訊

```
arm-none-eabi-gcc -g -o main.elf main.c syscalls.c -specs=nosys.specs -lc -lgcc
```

2. 啟動 GDB 並設置中斷點

```
(gdb) b main      # 設置 main 函數的中斷點（須先 target remote 連線）
```

1. 查看變數

```
(gdb) p variable  # 查看變數內容
```

4. 逐步執行程式

```
(gdb) s              # 逐步執行，進入函數內部
(gdb) n              # 逐步執行，不進入函數內部
```

5. 查看呼叫堆疊

```
(gdb) bt             # 顯示呼叫堆疊
```

6. 設置中斷點並反組譯函數

```
(gdb) b factorial          # 在函數 factorial 設置中斷點
(gdb) c                    # 繼續執行程式到中斷點
(gdb) disas factorial      # 查看函數的組合語言指令
```

IPSI Lab, Yuntech-CSIE

7. 逐步執行組合指令並檢查暫存器

```
(gdb) si                          # 單步執行一條機器指令
(gdb) info registers r0 r1        # 查看暫存器變化
```

8. 監控程序計數器（PC）

```
(gdb) display /i $pc              # 自動顯示當前 PC 指令
(gdb) si                          # 單步執行指令並更新顯示
```

IPSI Lab, Yuntech-CSIE

9. 逐步執行並檢查暫存器

```
(gdb) b add_numbers            # 設置中斷點
(gdb) c                        # 繼續執行至中斷點
(gdb) si                       # 單步執行機器指令
(gdb) info registers r0 r1     # 查看暫存器變化
```

10. 觀察記憶體內容

```
(gdb) x/4wx $sp                # 以 16 進位顯示 SP 指向的四個 32-bit word
(gdb) info registers r0        # 若 R0 是整數參數，直接看暫存器
(gdb) x/wx $r0                 # 只有 R0 存放有效位址時才解參考
```

IPSI Lab, Yuntech-CSIE

Arm GNU GDB 範例操作

GDB 的基礎使用範例

1. **接續第一個C語言範例。**
2. **編譯**

```
arm-none-eabi-gcc -g -o main.elf main.c syscalls.c -specs=nosys.specs -lc -lgcc
```

   - `-g`：加入除錯資訊。GDB 畫面若顯示 `(No debugging symbols found...)`，**先核對所載入的 ELF 是否就是此命令新產生的版本**，不要把不同輪建置的紀錄當成同一次實驗。

- 建議執行 `arm-none-eabi-readelf -S main.elf` 查看是否包含 `.debug_*` 區段，並檢查 `arm-none-eabi-gdb main.elf` 讀入的檔案路徑。

IPSI Lab, Yuntech-CSIE

啟用 GDB Server

3. 啟用QEMU與GDB Server

```
timeout 600s qemu-system-arm -M versatilepb -m 128M -nographic -kernel main.elf -S -s
```

- -S：啟動後立即暫停 QEMU，等待 GDB 連線。
- `-s`：建立 QEMU GDB stub（預設 TCP 1234；可用 `-gdb tcp::PORT` 指定），配合 `-S` 使 CPU 在連線前不執行。若擔心被其他主機連線，應限制監聽介面或使用防火牆。
- timeout 避免無限執行

IPSI Lab, Yuntech-CSIE

連入 GDB

4. **開啟另一Terminal，並移至相同目錄下，執行以下指令**

```
arm-none-eabi-gdb main.elf
# 啟用gdb模式

(gdb) target remote :1234
# 在gdb模式下
```

IPSI Lab, Yuntech-CSIE

EX.

```
lijw@DESKTOP-5B5R93E MINGW64 /c/users/lijw/sp/arm/c
$ arm-none-eabi-gdb main.elf
GNU gdb (Arm GNU Toolchain 13.3.Rel1 (Build arm-13.24)) 14.2.90.20240526-git
Copyright (C) 2023 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "--host=i686-w64-mingw32 --target=arm-none-eabi".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://bugs.linaro.org/>.
Find the GDB manual and other documentation resources online at:
    <http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from main.elf...
(gdb) target remote :1234
`C:\users\lijw\sp\arm\c\main.elf' has changed; re-reading symbols.
Remote debugging using :1234
0x000081dc in _start ()
(gdb)
```

程式執行進度

```
lijw@DESKTOP-5B5R93E MINGW64 /c/users/lijw/sp/arm/c
$ timeout 600s qemu-system-arm -M versatilepb -m 128M -nographic -kernel main.elf -S -s
Loop iteration: 0
Loop iteration: 1
Loop iteration: 2
Loop iteration: 3
```

gdb進度

```
lijw@DESKTOP-5B5R93E MINGW64 /c/users/lijw/sp/arm/c
$ arm-none-eabi-gdb main.elf
GNU gdb (Arm GNU Toolchain 13.3.Rel1 (Build arm-13.24)) 14.2.90.20240526-git
Copyright (C) 2023 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "--host=i686-w64-mingw32 --target=arm-none-eabi".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://bugs.linaro.org/>.
Find the GDB manual and other documentation resources online at:
    <http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from main.elf...
(gdb) target remote :1234
`C:\users\lijw\sp\arm\c\main.elf' has changed; re-reading symbols.
Remote debugging using :1234
0x000081dc in _start ()
(gdb) b main
warning: could not convert 'main' from the host encoding (CP1252) to UTF-32.
This normally should not happen, please file a bug report.
Breakpoint 1 at 0x8298: file main.c, line 5.
(gdb) c
Continuing.

Breakpoint 1, main () at main.c:5
5           for (i = 0; i < 10; i++) {
(gdb) p i
$1 = 0
(gdb) s
6               printf("Loop iteration: %d\n", i);
(gdb) s
5           for (i = 0; i < 10; i++) {
(gdb) s
6               printf("Loop iteration: %d\n", i);
(gdb) s
5           for (i = 0; i < 10; i++) {
(gdb) s
6               printf("Loop iteration: %d\n", i);
(gdb) s
5           for (i = 0; i < 10; i++) {
(gdb) s
6               printf("Loop iteration: %d\n", i);
(gdb) s
5           for (i = 0; i < 10; i++) {
(gdb) p i
$4 = 3
```

利用 GDB 查看暫存器內容

1. 接續C語言呼叫組合語言範例。
2. 編譯

```
arm-none-eabi-gcc -g -o main.elf main.c syscalls.c functions.s -specs=nosys.specs -lc -lgcc
```

啟用 GDB Server

3. 啟用QEMU與GDB

```
timeout 600s qemu-system-arm -M versatilepb -m 128M -nographic -kernel main.elf -S -s
```

IPSI Lab, Yuntech-CSIE

連入 GDB

4. **開啟另一Terminal，並移至相同目錄下，執行以下指令**

```
arm-none-eabi-gdb main.elf
# 啟用GDB模式

(gdb) target remote :1234
# 在GDB模式下
```

IPSI Lab, Yuntech-CSIE

逐步執行並檢查暫存器變化的流程

重要前提： 使用與 QEMU 端相同建置產物的 `main.elf`；先 `target remote :1234`，再設中斷點與 `continue`。若 breakpoint 位置為 `<factorial+8>`，代表已執行 `PUSH` 和 `MOV`，**不能當作函式入口尚未操作的狀態**。

1. **設置中斷點**
   停在 `add_numbers` 函數入口：

```
(gdb) b add_numbers
```

2. **執行程式**
   開始執行直到中斷點 `add_numbers`：

```
(gdb) c
```

IPSI Lab, Yuntech-CSIE

3. **檢查初始暫存器狀態**
   查看所有暫存器的值：

```
(gdb) info registers   # 縮寫 i r
```

4. **逐步執行並觀察變化**
   逐步執行，並在每步檢查暫存器：

```
(gdb) si          # 執行一條指令
(gdb) info registers
```

重複 `stepi` 和 `info registers`，直到函數執行結束。

IPSI Lab, Yuntech-CSIE

5. 檢查返回值

- 在函式準備返回的指令前，檢查返回值是否在 `r0`：

```
info registers
```

IPSI Lab, Yuntech-CSIE

```
lijw@DESKTOP-5B5R93E MINGW64 /c/users/lijw/sp/arm/ca2
$ arm-none-eabi-gdb main.elf
GNU gdb (Arm GNU Toolchain 13.3.Rel1 (Build arm-13.24)) 14.2.90.20240526-git
Copyright (C) 2023 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "--host=i686-w64-mingw32 --target=arm-none-eabi".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://bugs.linaro.org/>.
Find the GDB manual and other documentation resources online at:
    <http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from main.elf...
(No debugging symbols found in main.elf)   # 原始實驗紀錄；請核對 ELF 版本與 -g 選項
(gdb) target remote :1234
Remote debugging using :1234
0x000081dc in _start ()
(gdb) b add_numbers
Breakpoint 1 at 0x8588
(gdb) b factorial
Breakpoint 2 at 0x8598
(gdb) b swap_numbers
Breakpoint 3 at 0x85b4
```

IPSI Lab, Yuntech-CSIE

```
(gdb) c
Continuing.

Breakpoint 1, 0x00008588 in add_numbers ()
(gdb) i r
r0             0x3                 3
r1             0x7                 7
r2             0x0                 0
r3             0x0                 0
r4             0x0                 0
r5             0x0                 0
r6             0x0                 0
r7             0x0                 0
r8             0x0                 0
r9             0x0                 0
r10            0x60000             393216
r11            0x7affc             503804
r12            0x7aff0             503792
sp             0x7afe8             0x7afe8
lr             0x82a4              33444
pc             0x8588              0x8588 <add_numbers>
cpsr           0x600001d3          1610613203
fpscr          0x0                 0
fpsid          0x41011090          1090588816
fpexc          0x0                 0
DUMMY          0x0                 0
DBGDIDR        0x0                 0
MIDR           0x41069265          1090949733
CTR            0x1dd20d2           31269074
TCMTR          0x0                 0
TLBTR          0x0                 0
DUMMY          0x0                 0
JIDR           0x0                 0
DUMMY          0x0                 0
DACR           0x0                 0
TTBR0_EL1      0x0                 0
DFAR           0x0                 0
--Type <RET> for more, q to quit, c to continue without paging--q
Quit
```

IPSI Lab, Yuntech-CSIE

```
(gdb) si
0x000082a4 in main ()
(gdb) c
Continuing.

Breakpoint 2, 0x00008598 in factorial ()
(gdb) i r
# 注意：目前 PC = factorial+8（CMP 前）；本層 PUSH 與 MOV 已經執行
r0             0x5                 5
r1             0x18518             99608
r2             0x0                 0
r3             0x2889              10377
r4             0x5                 5
r5             0x0                 0
r6             0x0                 0
r7             0x0                 0
r8             0x0                 0
r9             0x0                 0
r10            0x60000             393216
r11            0x7affc             503804
r12            0x8b48              35656
sp             0x7afe0             0x7afe0
lr             0x82bc              33468
pc             0x8598              0x8598 <factorial+8>
cpsr           0x400001d3          1073742291
fpscr          0x0                 0
fpsid          0x41011090          1090588816
fpexc          0x0                 0
DUMMY          0x0                 0
DBGDIDR        0x0                 0
MIDR           0x41069265          1090949733
CTR            0x1dd20d2           31269074
TCMTR          0x0                 0
TLBTR          0x0                 0
DUMMY          0x0                 0
JIDR           0x0                 0
DUMMY          0x0                 0
DACR           0x0                 0
TTBR0_EL1      0x0                 0
DFAR           0x0                 0
--Type <RET> for more, q to quit, c to continue without paging--q
Quit
```

IPSI Lab, Yuntech-CSIE

```
(gdb) si
0x0000859c in factorial ()
(gdb) i r
r0             0x5                 5
r1             0x18518             99608
r2             0x0                 0
r3             0x2889              10377
r4             0x5                 5
r5             0x0                 0
r6             0x0                 0
r7             0x0                 0
r8             0x0                 0
r9             0x0                 0
r10            0x60000             393216
r11            0x7affc             503804
r12            0x8b48              35656
sp             0x7afe0             0x7afe0
lr             0x82bc              33468
pc             0x859c              0x859c <factorial+12>
cpsr           0x200001d3          536871379
fpscr          0x0                 0
fpsid          0x41011090          1090588816
fpexc          0x0                 0
DUMMY          0x0                 0
DBGDIDR        0x0                 0
MIDR           0x41069265          1090949733
CTR            0x1dd20d2           31269074
TCMTR          0x0                 0
TLBTR          0x0                 0
DUMMY          0x0                 0
JIDR           0x0                 0
DUMMY          0x0                 0
DACR           0x0                 0
TTBR0_EL1      0x0                 0
DFAR           0x0                 0
--Type <RET> for more, q to quit, c to continue without paging--q
Quit
(gdb) si
0x000085a0 in factorial ()
(gdb) si
0x000085a4 in factorial ()
(gdb) si
0x00008590 in factorial ()
(gdb) si
0x00008594 in factorial ()
(gdb) si

Breakpoint 2, 0x00008598 in factorial ()
```

IPSI Lab, Yuntech-CSIE

```
(gdb) i r
# 注意：目前 PC = factorial+8；遞迴下一層的 PUSH 後，SP 又降低 8 bytes
r0             0x4                 4
r1             0x18518             99608
r2             0x0                 0
r3             0x2889              10377
r4             0x4                 4
r5             0x0                 0
r6             0x0                 0
r7             0x0                 0
r8             0x0                 0
r9             0x0                 0
r10            0x60000             393216
r11            0x7affc             503804
r12            0x8b48              35656
sp             0x7afd8             0x7afd8
lr             0x85a8              34216
pc             0x8598              0x8598 <factorial+8>
cpsr           0x200001d3          536871379
fpscr          0x0                 0
fpsid          0x41011090          1090588816
fpexc          0x0                 0
DUMMY          0x0                 0
DBGDIDR        0x0                 0
MIDR           0x41069265          1090949733
CTR            0x1dd20d2           31269074
TCMTR          0x0                 0
TLBTR          0x0                 0
DUMMY          0x0                 0
JIDR           0x0                 0
DUMMY          0x0                 0
DACR           0x0                 0
TTBR0_EL1      0x0                 0
DFAR           0x0                 0
--Type <RET> for more, q to quit, c to continue without paging--q
Quit
```

IPSI Lab, Yuntech-CSIE

```
(gdb) c
Continuing.

Breakpoint 2, 0x00008598 in factorial ()
(gdb) c
Continuing.

Breakpoint 2, 0x00008598 in factorial ()
(gdb) c
Continuing.

Breakpoint 2, 0x00008598 in factorial ()
(gdb) c
Continuing.

Breakpoint 3, 0x000085b4 in swap_numbers ()
(gdb) i r
r0             0x7afec             503788
r1             0x7afe8             503784
r2             0x7afe8             503784
r3             0x7afec             503788
r4             0x0                 0
r5             0x0                 0
r6             0x0                 0
r7             0x0                 0
r8             0x0                 0
r9             0x0                 0
r10            0x60000             393216
r11            0x7affc             503804
r12            0x8b48              35656
sp             0x7afe8             0x7afe8
lr             0x8304              33540
pc             0x85b4              0x85b4 <swap_numbers>
cpsr           0x400001d3          1073742291
fpscr          0x0                 0
fpsid          0x41011090          1090588816
fpexc          0x0                 0
DUMMY          0x0                 0
DBGDIDR        0x0                 0
MIDR           0x41069265          1090949733
CTR            0x1dd20d2           31269074
TCMTR          0x0                 0
TLBTR          0x0                 0
DUMMY          0x0                 0
JIDR           0x0                 0
DUMMY          0x0                 0
DACR           0x0                 0
TTBR0_EL1      0x0                 0
DFAR           0x0                 0
--Type <RET> for more, q to quit, c to continue without paging--q
Quit
```

IPSI Lab, Yuntech-CSIE

其他：整數值與指標的差異

```
(gdb) x/xw $r0
0x0:    0x00000000      # 既有觀察紀錄；R0=0 時是在讀位址 0，不是整數值
(gdb) c
Continuing.

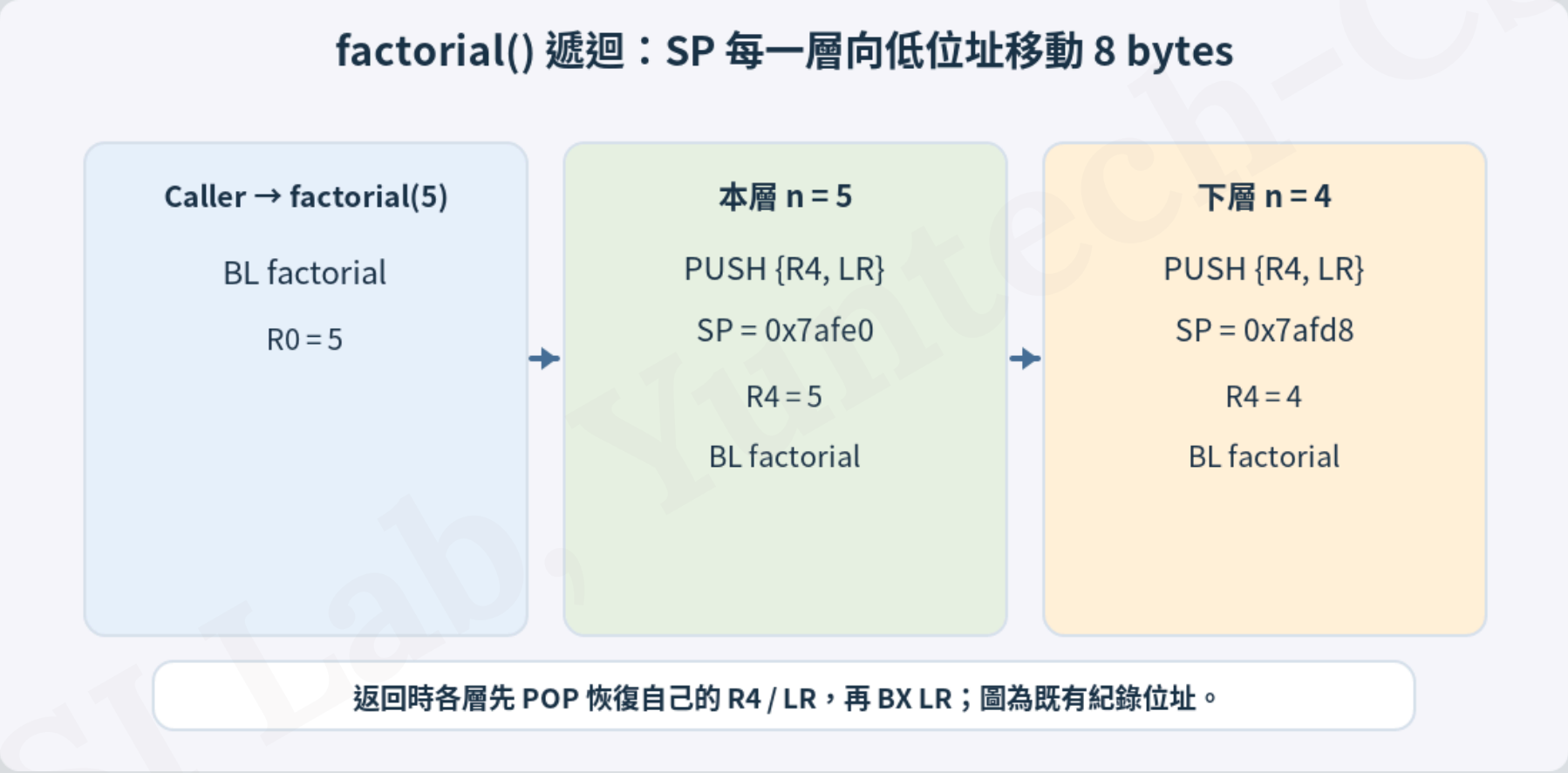
Breakpoint 1, 0x00008588 in add_numbers ()
(gdb) i r r0 r1
r0             0x3                 3
r1             0x7                 7
(gdb) si
0x0000858c in add_numbers ()
(gdb) i r r0 r1
r0             0xa                 10
r1             0x7                 7
(gdb) display /i $pc
1: x/i $pc
=> 0x858c <add_numbers+4>:      bx      lr
(gdb)
```

IPSI Lab, Yuntech-CSIE

此處是原始 GDB 觀察紀錄。`x/xw $r0` 會**把 R0 當位址來讀記憶體**，不是查看 R0 整數本身；只有在 `swap_numbers` 這類指標參數函式才可如此操作。觀察 `add_numbers` 整數值應使用 `info registers r0 r1`。

IPSI Lab, Yuntech-CSIE

追蹤 `factorial()`：每層都保存自己的 LR



對照既有紀錄： `factorial(5)` 時觀察到 SP `0x7afe0`，遞迴進入 `factorial(4)` 後為 `0x7afd8`（差 8 bytes）；兩次皆停在 `<factorial+8>`，已執行各自的 `PUSH {R4, LR}`。圖中以**相對堆疊框架**呈現，不假設所有函式都會有相同位址。

反組譯函數

```
(gdb) disas factorial
Dump of assembler code for function factorial:
   0x00008590 <+0>:     push    {r4, lr}
   0x00008594 <+4>:     mov     r4, r0
   0x00008598 <+8>:     cmp     r0, #1
   0x0000859c <+12>:    ble     0x85ac <fact_end>
   0x000085a0 <+16>:    sub     r0, r0, #1
   0x000085a4 <+20>:    bl      0x8590 <factorial>
   0x000085a8 <+24>:    mul     r0, r4, r0
   ...  (此處為原始紀錄節錄，尚未包含 fact_end 與返回指令)
End of assembler dump.
```

完整函式仍有 `fact_end`、`POP {R4, LR}`、`BX LR`。上方紀錄不是完整反組譯；要在 GDB 觀察後續指令，可用 `disassemble factorial` 或指定位址範圍再次檢查。